



BECE102L

Boolean Algebra

Basic Definitions

- Boolean algebra, like any other deductive mathematical system, may be defined with a set of elements, a set of operators, and a number of unproved axioms or postulates. If S is a set, and x and y are certain objects, then $x \in S$ denotes that x is a member of the set S , and $y \notin S$ denotes that y is not an element of S .
- The postulates of a mathematical system form the basic assumptions from which it is possible to deduce the rules, theorems, and properties of the system. The most common postulates used to formulate various algebraic structures are:

1. Closure. A set S is closed with respect to a binary operator if, for every pair of elements of S , the binary operator specifies a rule for obtaining a unique element of S . For example, the set of natural numbers $N = \{1, 2, 3, 4, \dots\}$ is closed with respect to the binary operator plus (+) by the rules of arithmetic addition, since for any $a, b \in N$ we obtain a unique $c \in N$ by the operation $a + b = c$.
2. Associative law. A binary operator $*$ on a set S is said to be associative whenever:

$$(x * y) * z = x * (y * z) \quad \text{for all } x, y, z \in S$$

3. Commutative law. A binary operator $*$ on a set S is said to be commutative whenever:

$$x * y = y * x \text{ for all } x, y \in S$$

4. Associative law. A binary operator $*$ on a set S is said to be associative whenever:

$$(x * y) * z = x * (y * z) \quad \text{for all } x, y, z \in S$$

5. Identity element. A set S is said to have an identity element with respect to a binary operation $*$ on S if there exists an element $e \in S$ with the property:

$$e * x = x * e = x \text{ for every } x \in S$$

6. Inverse. A set S having the identity element e with respect to a binary operator $*$ is said to have an inverse whenever, for every $x \in S$, there exists an element $y \in S$ such that:

$$x * y = e$$

5. Distributive law. If $*$ and $\bullet$ are two binary operators on a set S , $*$ is said to be distributive over $\bullet$ whenever:

$$x * (y \bullet z) = (x * y) \bullet (x * z)$$

AXIOMATIC DEFINITION OF BOOLEAN ALGEBRA

Boolean algebra is an algebraic structure defined by a set of elements, B , together with two binary operators, $+$ and $\cdot$, provided that the following (Huntington) postulates are satisfied:

1. (a) Closure with respect to the operator $+$.
(b) Closure with respect to the operator $\cdot$.
2. (a) An identity element with respect to $+$, designated by 0 : $x + 0 = 0 + x = x$.
(b) An identity element with respect to $\cdot$, designated by 1 : $x \cdot 1 = 1 \cdot x = x$.
3. (a) Commutative with respect to $+$: $x + y = y + x$.
(b) Commutative with respect to $\cdot$: $x \cdot y = y \cdot x$.
4. (a) $\cdot$ is distributive over $+$: $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$.
(b) $+$ is distributive over $\cdot$: $x + (y \cdot z) = (x + y) \cdot (x + z)$.
5. For every element $x \in B$, there exists an element $x' \in B$ (called the complement of x) such that: (a) $x + x' = 1$ and (b) $x \cdot x' = 0$.
6. There exists at least two elements $x, y \in B$ such that $x \neq y$.

Two-Valued Boolean Algebra

- A two-valued Boolean algebra is defined on a set of two elements, $B = \{0, 1\}$, with rules for the two binary operators $+$ and $\bullet$ as shown in the following operator tables

x	y	$x \bullet y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Basic Theorems and Properties of Boolean Algebra

1. Duality

- This theorem states that the dual of the Boolean function is obtained by interchanging the logical AND operator with logical OR operator and zeros with ones. For every Boolean function, there will be a corresponding Dual function.
- It states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. In a two-valued Boolean algebra, the identity elements and the elements of the set B are the same: 1 and 0.

Postulates and theorems of Boolean algebra

Postulate 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
Postulate 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x + x = x$	(b) $x \cdot x = x$
Theorem 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$
Theorem 3, involution	$(x')' = x$	
Postulate 3, commutative	(a) $x + y = y + x$	(b) $xy = yx$
Theorem 4, associative	(a) $x + (y + z) = (x + y) + z$	(b) $x(yz) = (xy) z$
Postulate 4, distributive	(a) $x (y + z) = xy + xz$	(b) $x + yz = (x + y) (x + z)$
Theorem 5, DeMorgan	(a) $(x + y)' = x' y'$	(b) $(xy)' = x' + y'$
Theorem 6, absorption	(a) $x + xy = x$	(b) $x(x + y) = x$

Basic Theorems

- The theorems and postulates are the most basic relationships in Boolean algebra. The theorems, like the postulates, are listed in pairs; each relation is the dual of the one paired with it. The postulates are basic axioms of the algebraic structure and need no proof. The theorems must be proven from the postulates.

THEOREM 1(a): $x + x = x$.

$$\begin{aligned}x + x &= (x + x) \bullet 1 && \text{by postulate: 2(b)} \\&= (x + x)(x + x') && 5(a) \\&= x + xx' && 4(b) \\&= x + 0 && 5(b) \\&= x && 2(a)\end{aligned}$$

THEOREM 1(b): $x \bullet x = x$.

$$\begin{aligned}x \bullet x &= xx + 0 && \text{by postulate: 2(a)} \\&= xx + xx' && 5(b) \\&= x(x + x') && 4(a) \\&= x \bullet 1 && 5(a) \\&= x && 2(b)\end{aligned}$$

THEOREM 2(a): $x + 1 = 1$.

$$\begin{aligned}x + 1 &= 1 \cdot (x + 1) && \text{by postulate: 2(b)} \\&= (x + x') (x + 1) && 5(a) \\&= x + x' \cdot 1 && 4(b) \\&= x + x' && 2(b) \\&= 1 && 5(a)\end{aligned}$$

THEOREM 2 (b): $x \cdot 0 = 0$ by duality.

THEOREM 3: $(x')' = x$.

THEOREM 6(a): $x + xy = x$.

$$\begin{aligned}x + xy &= x \cdot 1 + xy && \text{by postulate 2(b)} \\&= x (1 + y) && \text{by postulate 4(a)} \\&= x (y + 1) && \text{by postulate 3(a)} \\&= x \cdot 1 && \text{by theorem 2(a)} \\&= x && \text{by postulate 2(b)}\end{aligned}$$

THEOREM 6(b): $x(x + y) = x$ by duality.

De Morgan's theorem $(x + y)' = x' y'$

x	y	$x + y$	$(x + y)'$	x'	y'	$x' y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Operator Precedence

- The operator precedence for evaluating Boolean expressions is (1) parentheses, (2) NOT, (3) AND, and (4) OR.

EXAMPLE: Using basic Boolean theorem prove:

(a) $(x + y)(x + z) = x + yz$

Soln: $(x + y)(x + z)$

$$= x \cdot x + x \cdot z + y \cdot x + y \cdot z$$

since $x.x = x$

$$= x + xz + yx + yz$$

$$= x(1 + z) + yx + yz$$

since $(1 + z) = 1$

$$= x + yx + yz$$

$$= x(1 + y) + yz$$

since $(1 + y) = 1$

$$= x + yz \text{ (Proved)}$$

(b) $xy + xz + yz' = xz + yz'$

Soln: $xy + xz + yz'$

$$= xy(z + z') + xz(y + y') + yz'(x + x')$$

since $x + x' = 1$

$$= xyz + xyz' + xyz + xy'z + xyz' + x'yz'$$

$$= xyz + xyz' + xy'z + x'yz'$$

since $xyz + xyz = xyz$

$$= xyz + xy'z + xyz' + x'yz'$$

rearranging

$$= xz(y + y') + yz'(x + x')$$

since $y + y' = 1$ and $x + x' = 1$

$$= xz + yz' \text{ (Proved)}$$

EXAMPLE : Simplify the following Boolean functions to a minimum number of literals.

$$1. \quad x + x' y = (x + x')(x + y) = 1 \cdot (x + y) = x + y$$

$$2. \quad x (x' + y) = xx' + xy = 0 + xy = xy$$

$$3. \quad x' y' z + x' yz + xy' = x' z(y' + y) + xy' = x' z + xy'$$

$$\begin{aligned} 4. \quad xy + x' z + yz &= xy + x' z + yz(x + x') \\ &= xy + x' z + xyz + x' yz \\ &= xy(1 + z) + x' z(1 + y) \\ &= xy + x' z \end{aligned}$$

$$5. \quad (x + y)(x' + z)(y + z) = (x + y)(x' + z) \text{ by duality from function 4.}$$

Complement of a function

$$(A + B + C)' = (A + X)'$$

$$= A' X'$$

$$= A' \cdot (B + C)'$$

$$= A' \cdot (B' C')$$

$$= A' B' C'$$

$$\text{let } B + C = X$$

by theorem 5(a) (De Morgan)

substitute $B + C = X$

by theorem 5(a) (De Morgan)

by theorem 4(b) (associative)

$$(A + B + C + D + \dots + F)' = A' B' C' D' \dots F'$$

$$(ABCD \dots F)' = A' + B' + C' + D' + \dots + F'$$

- The generalized form of De Morgan's theorem states that the complement of a function is obtained by interchanging AND and OR operators and complementing each literal.

EXAMPLE : Find the complement of the functions $F_1 = x' yz' + x' y' z$ and $F_2 = x(y'z' + yz)$. Applying De Morgan's theorem as many times as necessary, the complements are obtained as follows:

$$F'_1 = (x' yz' + x' y' z)' = (x' yz')'(x' y' z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F'_2 &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')' \cdot (yz)' \\ &= x' + (y + z)(y' + z') \end{aligned}$$

Canonical and Standard Forms

Minterms and Maxterms

- A binary variable may appear either in its normal form (x) or in its complement form (x').
- For two binary variables x and y combined with an AND operation. Since each variable may appear in either form, there are four possible combinations: $x'y'$, $x'y$, xy' , and xy . Each of these four AND terms is called a minterm or a standard product.
- In a similar manner, n variables can be combined to form 2^n minterms. Each minterm is obtained from an AND term of the n variables.
- A symbol for each minterm is of the form m_j , where j denotes the decimal equivalent of the binary number of the minterm designated.

- In a similar fashion, n variables forming an OR term, provide 2^n possible combinations, called maxterms or standard sums.
- The eight maxterms for three variables, together with their symbolic designation are shown in the table.
- Any 2^n maxterms for n variables may be determined similarly. Each maxterm is obtained from an OR term of the n variables.*
Note that each maxterm is the complement of its corresponding minterm, and vice versa.

Minterms and maxterms for three binary variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Functions of three variables

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

$$f_1 = (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z)$$

$$= M_0 \cdot M_7 \cdot M_3 \cdot M_5 \cdot M_6$$

$$f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) = M_0 M_1 M_2 M_4$$

- Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form.

Sum of Minterms

- Any Boolean function can be expressed as a sum of minterms. The minterms whose sum defines the Boolean function are those that give the 1's of the function in a truth table.
- Since the function can be either 1 or 0 for each minterm and since there are 2^n minterms, one can calculate the possible functions that can be formed with n variables to be 2^n .
- It is sometimes convenient to express the Boolean function in its sum-of-minterms form.
- Each term is inspected to see if it contains all the variables. If it misses one or more variables, it is ANDed with an expression such as $x + x'$, where x is one of the missing variables.

EXAMPLE : Express the Boolean function $F = A + B'C$ in a sum of minterms. The function has three variables A , B , and C . The first term A is missing two variables; therefore:

$$A = A(B + B') = AB + AB'$$

This is still missing one variable:

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') \\ &= ABC + ABC' + AB'C + AB'C' \end{aligned}$$

The second term $B'C$ is missing one variable:

$$B'C = B'C(A + A') = AB'C + A'B'C$$

Combining all terms, we have:

$$\begin{aligned} F &= A + B'C \\ &= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C \end{aligned}$$

But $AB'C$ appears twice, and according to theorem 1 ($x + x = x$), it is possible to remove one of them. Rearranging the minterms in ascending order, we finally obtain:

$$\begin{aligned} F &= A'B'C + AB'C' + AB'C + ABC' + ABC \\ &= m_1 + m_4 + m_5 + m_6 + m_7 \end{aligned}$$

- It is sometimes convenient to express the Boolean function, when in its sum of minterms, in the following short notation:

$$F(A,B,C) = \Sigma (1,4, 5,6,7)$$

- The summation symbol Σ stands for the ORing of terms; the numbers following it are the minterms of the function. The letters in parentheses following F form a list of the variables in the order taken when the minterm is converted to an AND term.

Product of Maxterms

- Each of the 2^n functions of n binary variables can be also expressed as a product of maxterms.
- To express the Boolean function as a product of maxterms, it must first be brought into a form of OR terms. This may be done by using the distributive law $x + yz = (x + y)(x + z)$.
- Then any missing variable x in each OR term is ORed with xx' .

EXAMPLE : Express the Boolean function $F = xy + x'z$ in a product of maxterm form. First convert the function into OR terms using the distributive law:

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(y + x')(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

The function has three variables: x , y , and z . Each OR term is missing one variable; therefore:

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \end{aligned}$$

Combining all the terms and removing those that appear more than once, we finally obtain:

$$\begin{aligned} F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\ &= M_0 M_2 M_4 M_5 \end{aligned}$$

A convenient way to express this function is as follows:

$$F(x,y,z) = \prod(0,2, 4, 5)$$

Conversion between Canonical Forms

- The complement of a function expressed as the sum of minterms equals the sum of minterms missing from the original function. This is because the original function is expressed by those minterms that make the function equal to 1, while its complement is a 1 for those minterms that the function is a 0.

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

$$F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_2 + m_3$$

- Now, if we take the complement of F' by De Morgan's theorem, we obtain F in a different form:

$$F = (m_0 + m_2 + m_3)' = m'_0 \cdot m'_2 \cdot m'_3 = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

- To convert from one canonical form to another, interchange the symbols Σ and Π and list those numbers missing from the original form. As another example, the function:

$$F(x, y, z) = \Pi (0, 2, 4, 5)$$

- Can be expressed in the product of maxterm form. Its conversion to sum of minterms is:

$$F(x, y, z) = \Sigma (1, 3, 6, 7)$$

Boolean expressions for the 16 functions of two variables

Boolean functions	Operator symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exciusive-OR	x or y but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$x \odot y$	Equivalence*	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \supset y$	Implication	If y then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

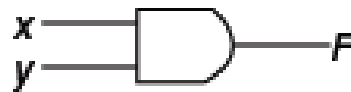
Standard form

- Another way to express Boolean functions is in standard form. In this configuration, the terms that form the function may contain one, two or any number of literals. There are two types of standard forms: the sum of products and product of sums.

Digital Logic Gates

Name	Graphic symbol	Algebraic function	Truth table
------	-------------------	-----------------------	----------------

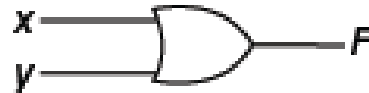
AND



$$F = xy$$

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

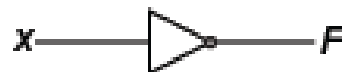
OR



$$F = x + y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

Inverter



$$F = x'$$

x	F
0	1
1	0

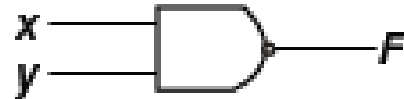
Buffer



$$F = x$$

x	F
0	0
1	1

NAND



$$F = (xy)'$$

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$F = (x + y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive-OR
(XOR)

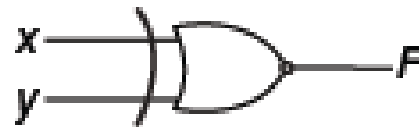


$$F = xy' + x'y$$

$$= x \oplus y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR
or
equivalence



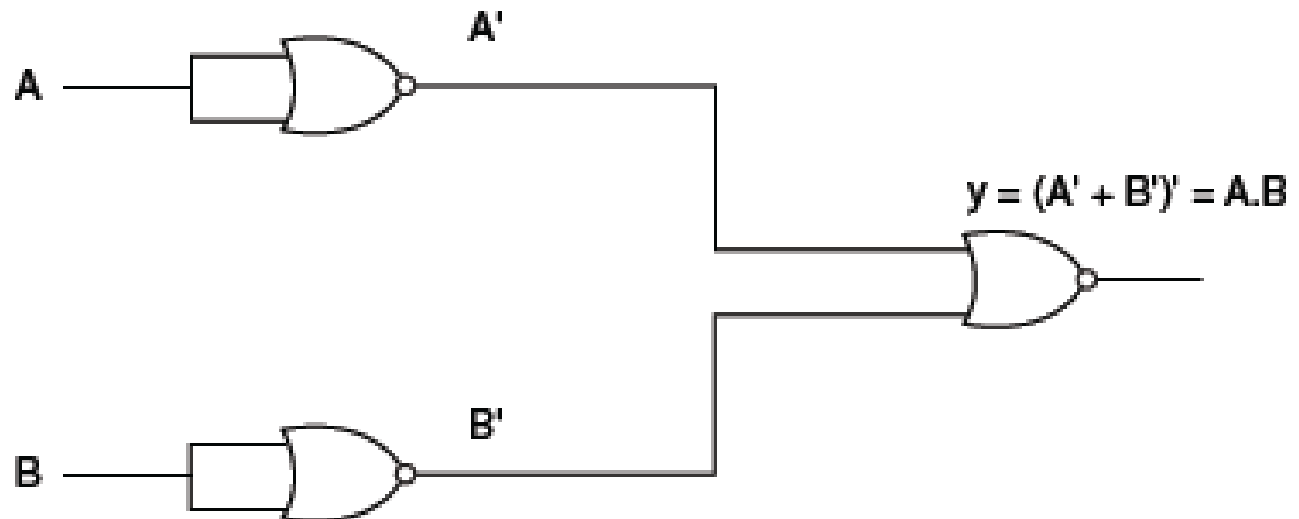
$$F = xy + x'y'$$

$$= x \odot y$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

EXAMPLE: Construct AND Gate using NOR Gate

Soln: $A.B = (A.B)'' = (A' + B')'$ using *D* Morgan's theorem



Simplification of Boolean Functions

- The complexity of the digital logic gates that implement a Boolean function is directly related to the complexity of the algebraic expression from which the function is implemented.
- Boolean functions may be simplified by algebraic means however, this procedure of minimization is awkward because it lacks specific rules to predict each succeeding step in the manipulative process.
- The map method provides a simple straightforward procedure for minimizing Boolean functions. This method may be regarded either as a pictorial form of a truth table or as an extension of the Venn diagram.
- The map method is also known as the “Veitch diagram” or the “Karnaugh map.”

K Maps

- An n -variable K-map has 2^n cells with each cell corresponding to an n -variable truth table value.
- K-map cells are labeled with the corresponding truth-table row.
- K-map cells are arranged such that adjacent cells correspond to truth rows that differ in only one bit position (logical adjacency).
- If m_i is a minterm of f , then place a 1 in cell i of the K-map.
- If M_i is a maxterm of f , then place a 0 in cell i .
- If d_i is a don't care of f , then place a d or x in cell i .

Simplification Guidelines for K-maps

- Always combine as many cells in a group as possible. This will result in the fewest number of literals in the term that represents the group.
- Make as few groupings as possible to cover all minterms. This will result in the fewest product terms.
- Always begin with the largest group, which means if you can find eight members group is better than two four groups and one four group is better than pair of two-group.

2 variable K-map: $f(a,b)$

$f(a,b)$		b	
		0	1
a	0	m_{00}	m_{01}
	1	m_{10}	m_{11}

OR

$f(a,b)$		a	
		0	1
b	0	m_{00}	m_{10}
	1	m_{01}	m_{11}

3-variable K-map: $f(a,b,c)$

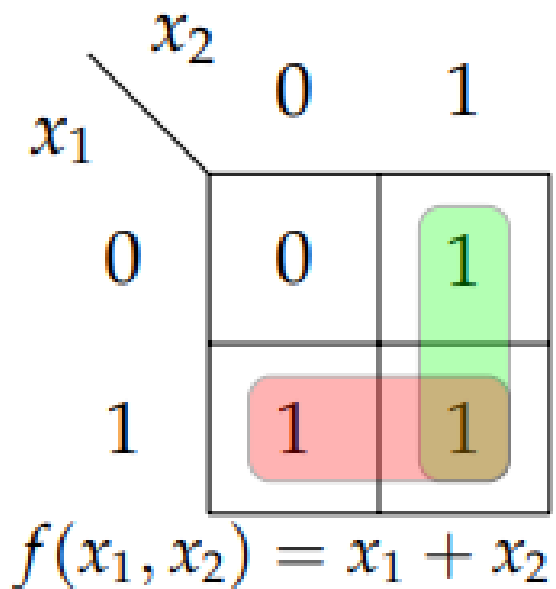
$f(a,b,c)$		bc				
		a	00	01	11	10
0	1	0	m_{000}	m_{001}	m_{011}	m_{010}
		1	m_{100}	m_{101}	m_{111}	m_{110}

4-variable K-map: $f(a,b,c,d)$

$f(a,b,c,d)$ ab \ cd		cd			
		00	01	11	10
ab	00	m_{0000}	m_{0001}	m_{0011}	m_{0010}
	01	m_{0100}	m_{0101}	m_{0111}	m_{0110}
	11	m_{1100}	m_{1101}	m_{1111}	m_{1110}
	10	m_{1000}	m_{1001}	m_{1011}	m_{1010}

Two variable Karnaugh maps

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1



- green group = $\bar{x}_1 x_2 + x_1 x_2 = x_1$
- red group = $x_1 \bar{x}_2 + x_1 x_2 = x_2$

Three variable Karnaugh maps

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Diagram illustrating the Karnaugh map for the function $f(x_1, x_2, x_3)$. The map is a 2x4 grid where the rows represent x_1 (0 and 1) and the columns represent x_2x_3 (00, 01, 11, 10). The function values are shown in the cells, with green highlighting for $x_1=0, x_2x_3 \in \{11, 10\}$ and gray highlighting for $x_1=1, x_2x_3 \in \{00, 01\}$.

	x_2x_3	00	01	11	10
x_1	0	0	0	1	1
	1	1	1	0	0

$$f(x_1, x_2, x_3) = x_1\bar{x}_2 + \bar{x}_1x_2$$

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

x_1 \ x_2x_3

	00	01	11	10
0	0	0	1	1
1	0	0	1	1

$$f(x_1, x_2, x_3) = x_2$$

Thumb Rules

1. No zeroes allowed.
2. No diagonal groupings
3. Only power of 2 cells in each grouping
4. Every one must be at least in one grouping
5. Overlapping is allowed
6. Groups may “Wrap-around”
7. Fewest number of groups possible

EXAMPLE : Simplify the Boolean function:

$$F = x'yz + x'yz' + xy'z' + xy'z$$

		00	01	11	10
x	0			1	1
x	1	1	1		

$$x'yz + x'yz' + xy'z' + xy'z = x'y + xy'$$

EXAMPLE : Simplify the Boolean function:

$$F = x'yz + xy'z' + xyz + xyz'$$

		yz		y	
		00	01	11	10
x	0			1	
x	1	1		1	1
		z			

$$x'yz + xy'z' + xyz + xyz' = xz + xz'$$

EXAMPLE : Simplify the Boolean function:

$$F = A'C + A'B + AB'C + BC$$

		B			
		BC		11	10
A	0			1	1
	1			1	1

$\underbrace{\hspace{10em}}_C$

$$A'C + A'B + AB'C + BC = C + A'B$$

EXAMPLE : Simplify the Boolean function;

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

		y			
		yz			
wx		00	01	11	10
w	00	1	1		1
	01	1	1		1
	11	1	1		1
	10	1	1		

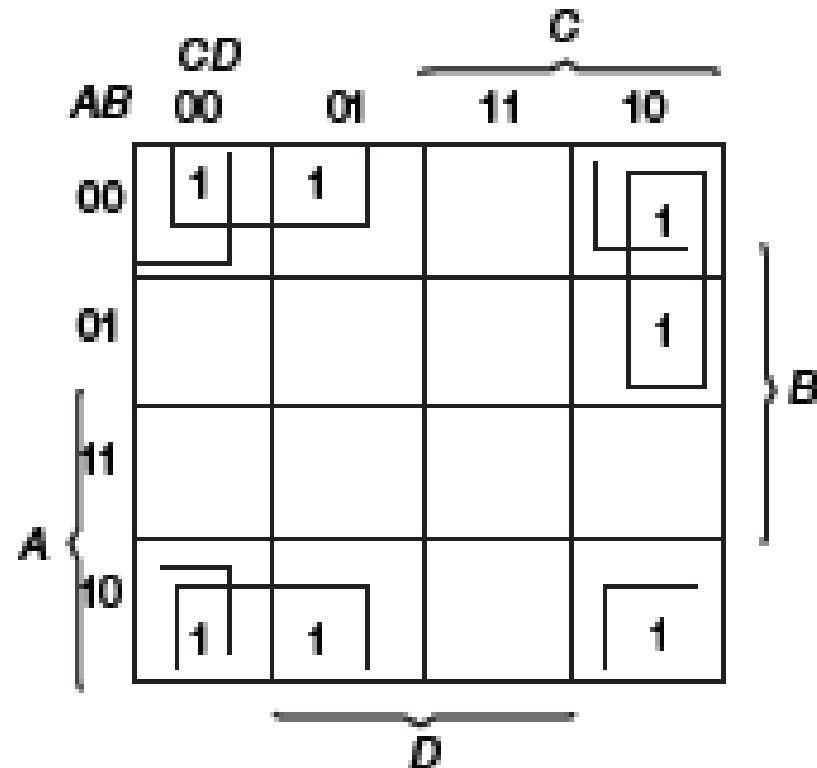
The Karnaugh map shows groupings for simplification:

- A group of four cells (00, 01, 11, 10) in the first two columns is grouped by a bracket labeled z .
- A group of four cells (00, 01, 11, 10) in the last two columns is grouped by a bracket labeled x .
- A group of four cells (00, 01, 11, 10) in the first two columns is grouped by a bracket labeled w .

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$$

EXAMPLE : Simplify the Boolean function:

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$



$$A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$$

Don't-care condition

EXAMPLE : Simplify the Boolean function:

$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

and the don't-care conditions:

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$

		y			
		yz			
wx		00	01	11	10
w	00	x	1	1	x
	01	0	x	1	0
	11	0	0	1	0
	10	0	0	1	0
		z			

Diagram illustrating a Karnaugh map for a function $F(w, x, y, z)$. The map is a 4x4 grid with rows labeled wx (00, 01, 11, 10) and columns labeled yz (00, 01, 11, 10). The map shows the following values:

- Row 00: x, 1, 1, x
- Row 01: 0, x, 1, 0
- Row 11: 0, 0, 1, 0
- Row 10: 0, 0, 1, 0

The map is partitioned into groups:

- A group of four cells (1s and Xs) is highlighted by a large rectangle, labeled $w'z$ on the left and z at the bottom.
- A group of four cells (1s and Xs) is highlighted by a large rectangle, labeled x on the right.
- Smaller rectangles highlight the 1s in the $w'z$ group.

(a) Combining 1's and X's $F = w'z + yz$

		y			
		yz			
		00	01	11	10
w	wz				
	00	$\overline{x}$	1	1	$\overline{x}$
	01	0	x	1	0
	11	0	0	1	0
	10	0	0	1	0

$\overbrace{\hspace{1.5cm}}^y$
 $\underbrace{\hspace{1.5cm}}_z$
 $\left. \begin{array}{c} \text{ } \\ \text{ } \\ \text{ } \\ \text{ } \end{array} \right\} w$
 $\left. \begin{array}{c} \text{ } \\ \text{ } \\ \text{ } \\ \text{ } \end{array} \right\} x$

(b) Combining 0's and X's $F = z(w' + y)$

Simplify by *K*-Map

1. Simplify the following equation by *K*-Map

$$F(A, B, C) = \sum m(0, 2, 4) + \sum d(1, 3, 5, 6, 7)$$

<i>BC</i> <i>A</i>		<i>BC</i>			
		00	01	11	10
0	0	1 ⁰	X ¹	X ³	1 ²
1	1	X ⁴	X ⁵	X ⁷	X ⁶

$$F(A, B, C) = C'$$

2. Simplify $\pi(0, 2, 3, 6)$ by K -Map

BC					
		00	01	11	10
A	0	0	1	3	2
	1	4	5	7	6

$$f = (A + C) (A + B') (B' + C)$$

3. Simplify the following equation by *K*-Map
- $$F(A, B, C, D) = \sum m(2, 4, 5, 13, 14) + \sum d(0, 1, 8, 10)$$

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	0 X	1 X	3	2 1
	01	4 1	5 1	7	6
	11	12	13 1	15	14 1
	10	8	9	11	10 X

Ans. $F = A'C' + BC'D + B'CD' + ACD'$

THANK YOU